

✓ Intrusion Detection System using Machine Learning

AIML Assignment 2

Objective: Build and compare multiple Machine Learning models for Network Intrusion Detection using the KDD Cup 1999 dataset.

Dataset: KDD Cup 1999 — a benchmark dataset for intrusion detection containing ~4.9 million network connection records with 41 features and a label indicating normal or attack type.

Attack Categories:

Category	Description	Examples
DoS	Denial of Service	smurf, neptune, back, teardrop, pod, land
Probe	Surveillance/Probing	portsweep, ipsweep, nmap, satan
R2L	Remote to Local	warezclient, guess_passwd, warezmaster, ftp_write
U2R	User to Root	buffer_overflow, rootkit, loadmodule, perl
Normal	Legitimate traffic	normal

Models Implemented:

1. Naive Bayes
2. Decision Tree
3. Random Forest
4. Support Vector Machine (SVM)
5. Logistic Regression
6. Gradient Boosting

✓ 1. Environment Setup & Data Loading

```
# Install kagglehub if not already installed  
!pip install -q kagglehub
```

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import glob

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler
from sklearn.feature_selection import SelectKBest, mutual_info_classif, chi2
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve
)

from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

import time

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 12

print("All libraries imported successfully.")
```

All libraries imported successfully.

```
import kagglehub

# Download latest version of KDD Cup 1999 dataset
path = kagglehub.dataset_download("kav131/kdd-cup-1999-data")
print("Path to dataset files:", path)

# List files in the downloaded directory
for f in os.listdir(path):
    fpath = os.path.join(path, f)
    size_mb = os.path.getsize(fpath) / (1024 * 1024)
    print(f" {f} - {size_mb:.1f} MB")
```

Using Colab cache for faster access to the 'kdd-cup-1999-data' dataset.
Path to dataset files: /kaggle/input/kdd-cup-1999-data

- binary_classification - 0.0 MB
- LICENSE - 0.0 MB
- README.md - 0.0 MB
- unlabeled_data - 0.0 MB
- full_multiclass - 0.0 MB
- reduced_multiclass - 0.0 MB
- all_attack_types.txt - 0.0 MB

```
# Debug: inspect what kagglehub returned
print(f"Path returned: {path}")
print(f"Is file: {os.path.isfile(path)}")
print(f"Is dir: {os.path.isdir(path)}")

# If path is a file, use it directly
if os.path.isfile(path):
    data_file = path
    print(f"\nPath is a file – using directly: {path}")
else:
    # Walk the directory tree to find ALL files (including nested)
    all_files = []
    for root, dirs, files in os.walk(path):
        for f in files:
            fpath = os.path.join(root, f)
            size_mb = os.path.getsize(fpath) / (1024 * 1024)
            all_files.append((f, fpath, size_mb))
            print(f" {os.path.relpath(fpath, path)} – {size_mb:.1f} MB")

    if len(all_files) == 0:
        raise FileNotFoundError(f"No files found under {path}. Check kagglehub download.")

    # Pick the largest file
    all_files.sort(key=lambda x: x[2], reverse=True)
    data_file = all_files[0][1]
    print(f"\nUsing: {all_files[0][0]} ({all_files[0][2]:.1f} MB)")

# Read a small preview to understand structure
preview = pd.read_csv(data_file, nrows=5)
print(f"\nPreview – shape: {preview.shape}")
print(f"Columns: {preview.columns.tolist()}")
print(preview.head())

# Load full dataset
df = pd.read_csv(data_file)
print(f"\nDataset loaded: {df.shape[0]:,} rows × {df.shape[1]} columns")
print(f"Columns ({len(df.columns)}): {df.columns.tolist()}")
```

```
reduced_multiclass/reduced_multiclass/cleaned/full_5_classes_cleaned.csv - 625.4 MB
reduced_multiclass/reduced_multiclass/cleaned/test_5_classes_cleaned.csv - 39.7 MB
reduced_multiclass/reduced_multiclass/cleaned/train_5_classes_cleaned.csv - 63.1 MB
reduced_multiclass/reduced_multiclass/raw/test_5_classes.csv - 40.3 MB
reduced_multiclass/reduced_multiclass/raw/train_5_classes.csv - 64.0 MB
reduced_multiclass/reduced_multiclass/raw/full_5_classes.csv - 634.7 MB
```

Using: full_binary.csv (645.8 MB)

Preview – shape: (5, 42)

Columns: ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent']

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	\
0	0	tcp	http	SF	215	45076	0	
1	0	tcp	http	SF	162	4528	0	
2	0	tcp	http	SF	236	1228	0	
3	0	tcp	http	SF	233	2032	0	

0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

	dst_host_srv_error_rate	connection_type
0	0.0	normal
1	0.0	normal
2	0.0	normal
3	0.0	normal
4	0.0	normal

[5 rows x 42 columns]

Dataset loaded: 4,898,431 rows x 42 columns

Columns (42): ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'window_bytes', 'un...

2. Exploratory Data Analysis (EDA)

```
# Basic info
print("=" * 60)
print("DATASET OVERVIEW")
print("=" * 60)
print(f"Shape: {df.shape}")
print(f"\nData types:\n{df.dtypes.value_counts()}")
print(f"\nMemory usage: {df.memory_usage(deep=True).sum() / 1e6:.1f} MB")
print(f"\nFirst 5 rows:")
df.head()
```

```
=====
DATASET OVERVIEW
=====
Shape: (4898431, 42)
```

```
Data types:
int64      23
float64    15
object     4
Name: count, dtype: int64
```

```
Memory usage: 2532.5 MB
```

```
First 5 rows:
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_cou
0	0	tcp	http	SF	215	45076	0	0	0	0	...	
1	0	tcp	http	SF	162	4528	0	0	0	0	...	
2	0	tcp	http	SF	236	1228	0	0	0	0	...	
3	0	tcp	http	SF	233	2032	0	0	0	0	...	
4	0	tcp	http	SF	239	486	0	0	0	0	...	

```
5 rows × 42 columns
```

```
# Statistical summary of numerical features
df.describe().T
```


	count	mean	std	min	25%	50%	75%	max
duration	4898431.0	4.834243e+01	723.329811	0.0	0.00	0.0	0.00	5.832900e+04
src_bytes	4898431.0	1.834621e+03	941431.074484	0.0	45.00	520.0	1032.00	1.379964e+09
dst_bytes	4898431.0	1.093623e+03	645012.333754	0.0	0.00	0.0	0.00	1.309937e+09
land	4898431.0	5.716116e-06	0.002391	0.0	0.00	0.0	0.00	1.000000e+00
wrong_fragment	4898431.0	6.487792e-04	0.042854	0.0	0.00	0.0	0.00	3.000000e+00
urgent	4898431.0	7.961733e-06	0.007215	0.0	0.00	0.0	0.00	1.400000e+01
hot	4898431.0	1.243766e-02	0.468978	0.0	0.00	0.0	0.00	7.700000e+01
num_failed_logins	4898431.0	3.205108e-05	0.007299	0.0	0.00	0.0	0.00	5.000000e+00
logged_in	4898431.0	1.435290e-01	0.350612	0.0	0.00	0.0	0.00	1.000000e+00
num_compromised	4898431.0	8.088304e-03	3.856481	0.0	0.00	0.0	0.00	7.479000e+03
root_shell	4898431.0	6.818510e-05	0.008257	0.0	0.00	0.0	0.00	1.000000e+00
su_attempted	4898431.0	3.674646e-05	0.008082	0.0	0.00	0.0	0.00	2.000000e+00
num_root	4898431.0	1.293496e-02	3.938075	0.0	0.00	0.0	0.00	7.468000e+03
num_file_creations	4898431.0	1.188748e-03	0.124186	0.0	0.00	0.0	0.00	4.300000e+01
num_shells	4898431.0	7.430951e-05	0.008738	0.0	0.00	0.0	0.00	2.000000e+00



```

# Identify the target / label column
# This Kaggle version uses 'connection_type' as the target label
# It may also have a 'label' column with 5-class or binary labels
print("Candidate label columns and their unique values:")
for col in df.columns:
    if df[col].dtype == 'object':
        nuniq = df[col].nunique()
        print(f" {col}: {nuniq} unique – {df[col].unique()[0:10]}")

# Use 'connection_type' as the main label if it exists, otherwise fall back to 'label'
if 'connection_type' in df.columns:
    label_col = 'connection_type'
elif 'label' in df.columns:
    label_col = df.columns[-1] # last column is typically the target
else:
    label_col = df.columns[-1]

print(f"\nUsing '{label_col}' as the target column.")

# Ensure label column is string type and clean it
df[label_col] = df[label_col].astype(str).str.strip().str.rstrip('.')

# Distribution of labels
print(f"\nLabel distribution ({label_col}):")
label_counts = df[label_col].value_counts()
print(label_counts)
print(f"\nTotal unique labels: {df[label_col].nunique()}")

```

```

Candidate label columns and their unique values:
protocol_type: 3 unique – ['tcp' 'udp' 'icmp']
service: 70 unique – ['http' 'smtp' 'domain_u' 'auth' 'finger' 'telnet' 'eco_i' 'ftp' 'ntp_u'
'ecr_i']
flag: 11 unique – ['SF' 'S2' 'S1' 'S3' 'OTH' 'REJ' 'RSTO' 'S0' 'RSTR' 'RSTOS0']
connection_type: 2 unique – ['normal' 'attack']

```

Using 'connection_type' as the target column.

Label distribution (connection_type):

```
connection_type
attack      3925650
normal      972781
Name: count, dtype: int64
```

```
Total unique labels: 2
```

```
# Map specific attack types to 5 broader categories
attack_map = {
    'normal': 'Normal',
    # DoS attacks
    'back': 'DoS', 'land': 'DoS', 'neptune': 'DoS', 'pod': 'DoS',
    'smurf': 'DoS', 'teardrop': 'DoS', 'mailbomb': 'DoS', 'processtable': 'DoS',
    'udpstorm': 'DoS', 'apache2': 'DoS', 'worm': 'DoS',
    # Probe attacks
    'ipsweep': 'Probe', 'nmap': 'Probe', 'portsweep': 'Probe', 'satan': 'Probe',
    'mscan': 'Probe', 'saint': 'Probe',
    # R2L attacks
    'ftp_write': 'R2L', 'guess_passwd': 'R2L', 'imap': 'R2L', 'multihop': 'R2L',
    'phf': 'R2L', 'spy': 'R2L', 'warezclient': 'R2L', 'warezmaster': 'R2L',
    'sendmail': 'R2L', 'named': 'R2L', 'snmpgetattack': 'R2L', 'snmpguess': 'R2L',
    'xlock': 'R2L', 'xsnoop': 'R2L', 'httptunnel': 'R2L',
    # U2R attacks
    'buffer_overflow': 'U2R', 'loadmodule': 'U2R', 'perl': 'U2R',
    'rootkit': 'U2R', 'xterm': 'U2R', 'ps': 'U2R', 'sqlattack': 'U2R',
    # Pre-mapped 5-class labels (this Kaggle version may already use these)
    'dos': 'DoS', 'probe': 'Probe', 'r2l': 'R2L', 'u2r': 'U2R',
    # Binary label
    'attack': 'Attack',
}

# Check if the dataset already has a 5-class or binary label structure
unique_labels = set(df[label_col].str.lower().unique())
five_class = {'normal', 'dos', 'probe', 'r2l', 'u2r'}
is_already_5class = unique_labels.issubset(five_class)

if is_already_5class:
    print("Dataset already has 5-class labels – mapping directly.")
    df['attack_category'] = df[label_col].str.lower().map(attack_map)
else:
    print("Dataset has fine-grained labels – mapping to 5 categories.")
    df['attack_category'] = df[label_col].str.lower().map(attack_map)

df['attack_category'] = df['attack_category'].fillna('Unknown')
```

```
print("\nAttack Category Distribution:")
cat_counts = df['attack_category'].value_counts()
print(cat_counts)

unknown_labels = df[df['attack_category'] == 'Unknown'][label_col].unique()
if len(unknown_labels) > 0:
    print(f"\nUnmapped labels: {unknown_labels}")
else:
    print("\nAll labels mapped successfully.")
```

Dataset has fine-grained labels – mapping to 5 categories.

Attack Category Distribution:

```
attack_category
Attack      3925650
Normal      972781
Name: count, dtype: int64
```

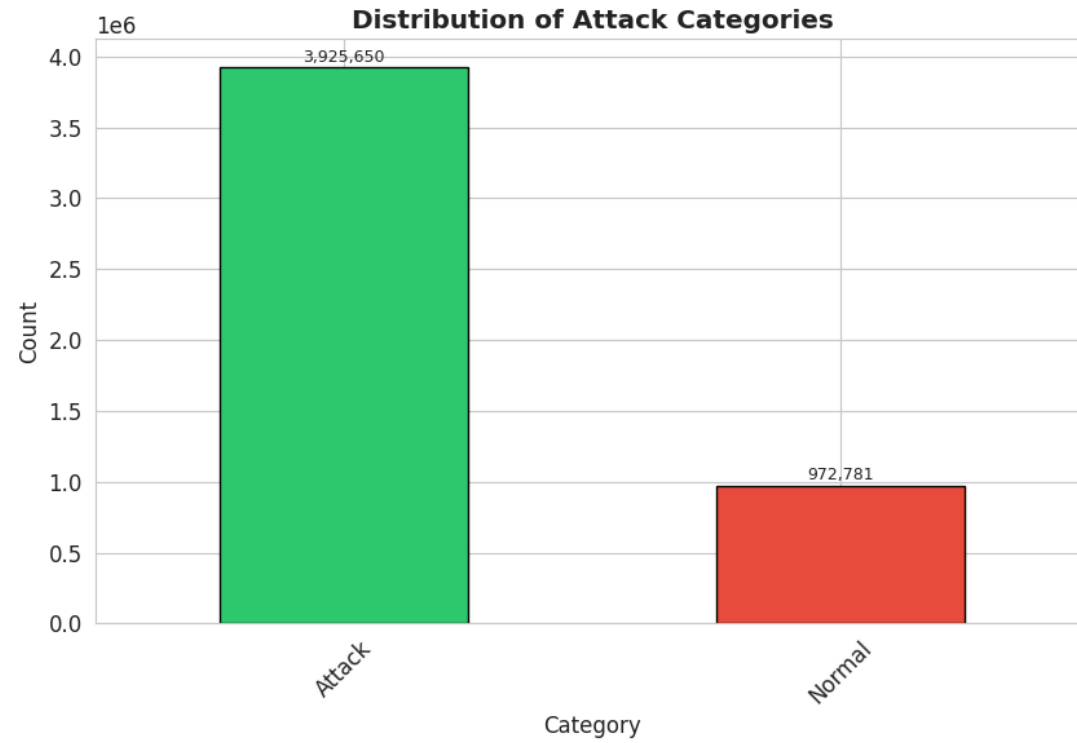
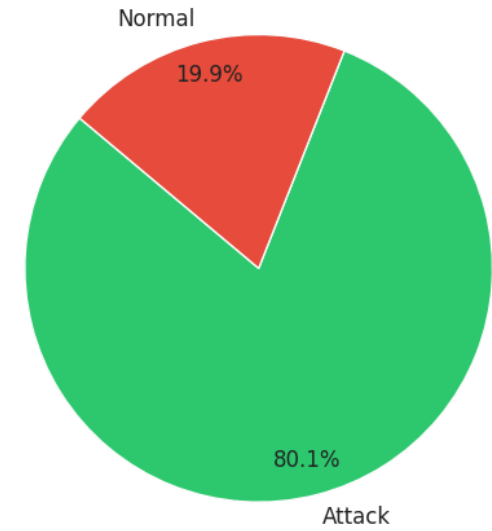
All labels mapped successfully.

```
# Visualize attack category distribution
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Bar chart
colors = ['#2ecc71', '#e74c3c', '#3498db', '#f39c12', '#9b59b6']
cat_counts.plot(kind='bar', ax=axes[0], color=colors[:len(cat_counts)], edgecolor='black')
axes[0].set_title('Distribution of Attack Categories', fontweight='bold')
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Count')
axes[0].tick_params(axis='x', rotation=45)
for i, v in enumerate(cat_counts.values):
    axes[0].text(i, v + max(cat_counts)*0.01, f'{v:,}', ha='center', fontsize=9)

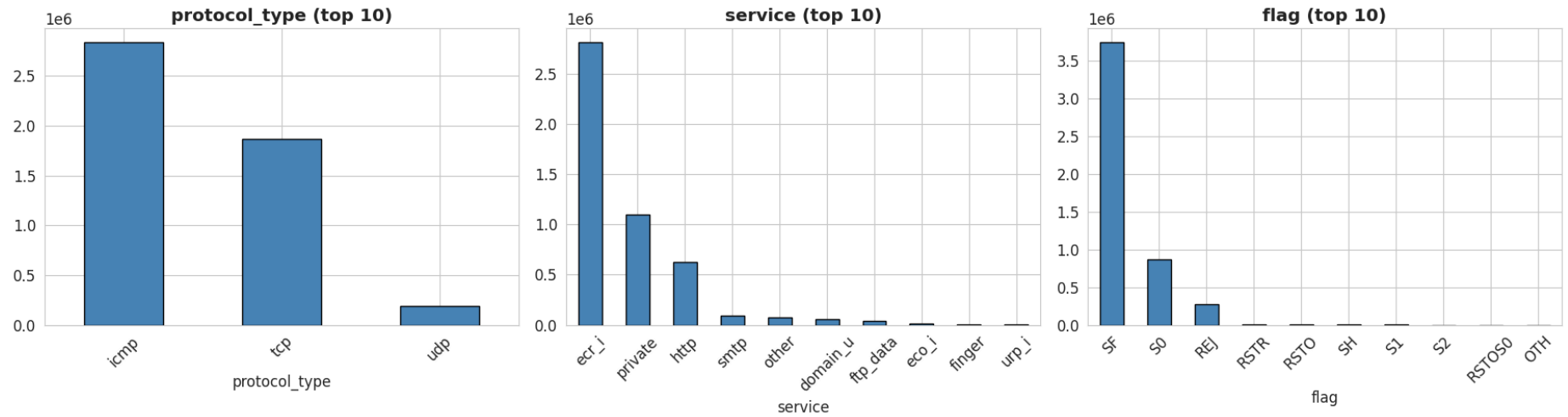
# Pie chart
axes[1].pie(cat_counts.values, labels=cat_counts.index, autopct='%1.1f%%',
            colors=colors[:len(cat_counts)], startangle=140, pctdistance=0.85)
axes[1].set_title('Proportion of Attack Categories', fontweight='bold')

plt.tight_layout()
plt.show()
```

**Proportion of Attack Categories**


```
# Distribution of categorical features
cat_features = [col for col in df.select_dtypes(include=['object']).columns
                 if col not in [label_col, 'attack_category']]

if len(cat_features) > 0:
    n_plots = min(len(cat_features), 3)
    fig, axes = plt.subplots(1, n_plots, figsize=(6 * n_plots, 5))
    if n_plots == 1:
        axes = [axes]
    for i, feat in enumerate(cat_features[:n_plots]):
        top_vals = df[feat].value_counts().head(10)
        top_vals.plot(kind='bar', ax=axes[i], color='steelblue', edgecolor='black')
        axes[i].set_title(f'{feat} (top 10)', fontweight='bold')
        axes[i].tick_params(axis='x', rotation=45)
    plt.tight_layout()
    plt.show()
else:
    print("No categorical features to plot (all may already be encoded).")
```



3. Data Pre-processing

```
# Check for missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
missing_info = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct})
print("Missing values per column:")
print(missing_info[missing_info['Missing Count'] > 0])
if missing.sum() == 0:
    print("No missing values found in the dataset.")
```

```
Missing values per column:
Empty DataFrame
Columns: [Missing Count, Missing %]
Index: []
No missing values found in the dataset.
```

```
# Check for duplicate rows
dup_count = df.duplicated().sum()
print(f"Duplicate rows: {dup_count:,} ({dup_count/len(df)*100:.2f}%)")

# Remove duplicates
df = df.drop_duplicates()
print(f"Shape after removing duplicates: {df.shape}")
```

```
Duplicate rows: 3,823,448 (78.05%)
Shape after removing duplicates: (1074983, 43)
```

```
# Create binary label: Normal vs Attack
df['binary_label'] = df['attack_category'].apply(lambda x: 0 if x == 'Normal' else 1)

print("Binary label distribution:")
print(df['binary_label'].value_counts().rename({0: 'Normal', 1: 'Attack'}))
print(f"\nAttack ratio: {df['binary_label'].mean():.2%}")
```

```
Binary label distribution:
binary_label
Normal      812814
Attack      262169
```

Name: count, dtype: int64

Attack ratio: 24.39%

```
# Encode categorical features using LabelEncoder
label_encoders = {}
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
# Exclude label/target columns from encoding
exclude_cols = [label_col, 'attack_category', 'label', 'connection_type']
categorical_cols = [c for c in categorical_cols if c not in exclude_cols]

if len(categorical_cols) > 0:
    print("Encoding categorical features:")
    for col in categorical_cols:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        label_encoders[col] = le
        print(f" {col}: {len(le.classes_)} unique values")
    print("\nAll categorical features encoded.")
else:
    print("No categorical features to encode (dataset may already be numeric).")
```

```
Encoding categorical features:
protocol_type: 3 unique values
service: 70 unique values
flag: 11 unique values
```

All categorical features encoded.

```
# Prepare feature matrix and target
# Drop all label/target/non-feature columns
drop_cols = [label_col, 'attack_category', 'binary_label', 'label', 'connection_type']
drop_cols = [c for c in drop_cols if c in df.columns]

# Also drop any remaining object-type columns (shouldn't be any after encoding)
remaining_obj = df.select_dtypes(include=['object']).columns.tolist()
drop_cols = list(set(drop_cols + remaining_obj))

feature_cols = [c for c in df.columns if c not in drop_cols]
X = df[feature_cols].copy()
y = df['binary_label'].copy() # Binary classification: Normal (0) vs Attack (1)

print(f"Dropped columns: {drop_cols}")
print(f"Feature matrix shape: {X.shape}")
print(f"Features: {X.columns.tolist()}")
print(f"\nTarget distribution:\n{y.value_counts().rename({0: 'Normal', 1: 'Attack'})}")
```

```
Dropped columns: ['binary_label', 'attack_category', 'connection_type']
Feature matrix shape: (1074983, 41)
Features: ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent',

Target distribution:
binary_label
Normal      812814
Attack      262169
Name: count, dtype: int64
```

```
# Sampling for computational efficiency
# The full dataset can be very large; use stratified sampling if needed
SAMPLE_SIZE = 100000 # Adjust based on available resources

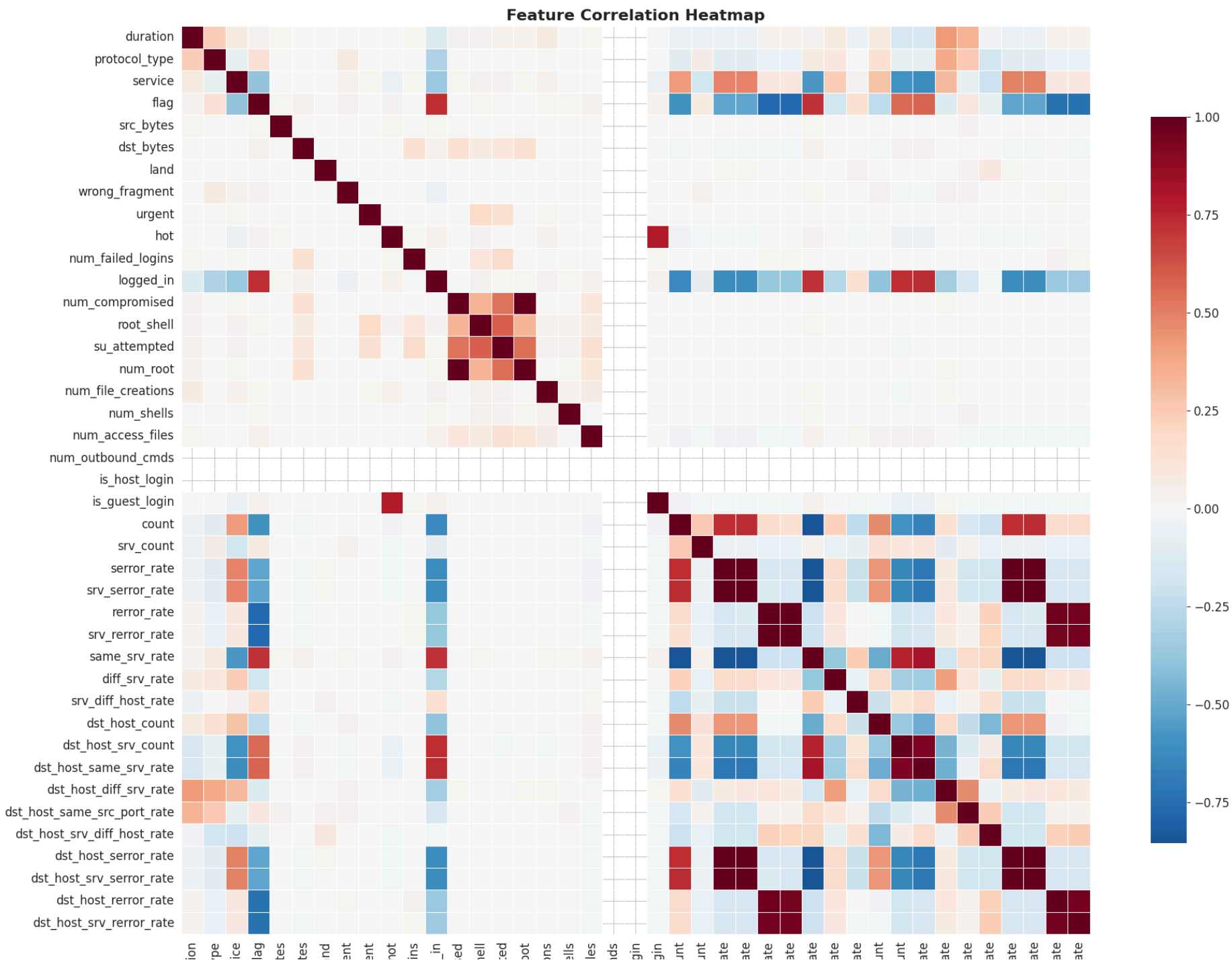
if len(X) > SAMPLE_SIZE:
    print(f"Dataset is large ({len(X):,} rows). Using stratified sample of {SAMPLE_SIZE:,} rows.")
    X_sampled, _, y_sampled, _ = train_test_split(
        X, y, train_size=SAMPLE_SIZE, stratify=y, random_state=42
    )
    X = X_sampled.reset_index(drop=True)
    y = y_sampled.reset_index(drop=True)
    print(f"Sampled shape: {X.shape}")
    print(f"Sampled target distribution:\n{y.value_counts().rename({0: 'Normal', 1: 'Attack'})}")
else:
    print(f"Dataset size ({len(X):,}) is manageable. Using full dataset.")
```

```
Dataset is large (1,074,983 rows). Using stratified sample of 100,000 rows.
Sampled shape: (100000, 41)
Sampled target distribution:
binary_label
Normal      75612
Attack      24388
Name: count, dtype: int64
```

✓ 4. Data Correlation Analysis

```
# Correlation matrix
corr_matrix = X.corr()

plt.figure(figsize=(20, 16))
sns.heatmap(corr_matrix, cmap='RdBu_r', center=0, linewidths=0.5,
            fmt='.1f', square=True, cbar_kws={'shrink': 0.8})
plt.title('Feature Correlation Heatmap', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

```

durati
protocol_ty
serv
f
src_byt
dst_byt
la
wrong_fragme
urg€
l
num_failed_log
logged
num_compromis
root_sh
su_attempt
num_rc
num_file_creati
num_sh€
num_access_fi
num_outbound_cm
is_host_loi
is_guest_loi
col
srv_col
seerror_r
srv_seerror_r
error_r
srv_reerror_r
same_srv_r
diff_srv_r
srv_diff_host_r
dst_host_col
dst_host_srv_col
dst_host_same_srv_r
dst_host_diff_srv_r
dst_host_same_src_port_r
dst_host_srv_diff_host_r
dst_host_seerror_r
dst_host_srv_seerror_r
dst_host_reerror_r
dst_host_srv_reerror_r

```

```

# Find highly correlated feature pairs (|r| > 0.9)
high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > 0.9:
            high_corr_pairs.append({
                'Feature 1': corr_matrix.columns[i],
                'Feature 2': corr_matrix.columns[j],
                'Correlation': corr_matrix.iloc[i, j]
            })

high_corr_df = pd.DataFrame(high_corr_pairs).sort_values('Correlation', ascending=False)
print(f"Highly correlated pairs (|r| > 0.9): {len(high_corr_df)}")
high_corr_df

```

Highly correlated pairs ($|r| > 0.9$): 14

	Feature 1	Feature 2	Correlation	
5	srv_serror_rate	dst_host_srv_serror_rate	0.997497	
1	serror_rate	srv_serror_rate	0.996914	
0	num_compromised	num_root	0.996806	
2	serror_rate	dst_host_serror_rate	0.995519	
3	serror_rate	dst_host_srv_serror_rate	0.994980	
12	dst_host_serror_rate	dst_host_srv_serror_rate	0.994896	
4	srv_serror_rate	dst_host_serror_rate	0.993386	
6	rerror_rate	srv_rerror_rate	0.989409	
7	rerror_rate	dst_host_rerror_rate	0.966035	
13	dst_host_rerror_rate	dst_host_srv_rerror_rate	0.961505	
10	srv_rerror_rate	dst_host_srv_rerror_rate	0.958601	
8	rerror_rate	dst_host_srv_rerror_rate	0.957677	
9	srv_rerror_rate	dst_host_rerror_rate	0.956288	
11	dst_host_srv_count	dst_host_same_srv_rate	0.931362	

Next steps:

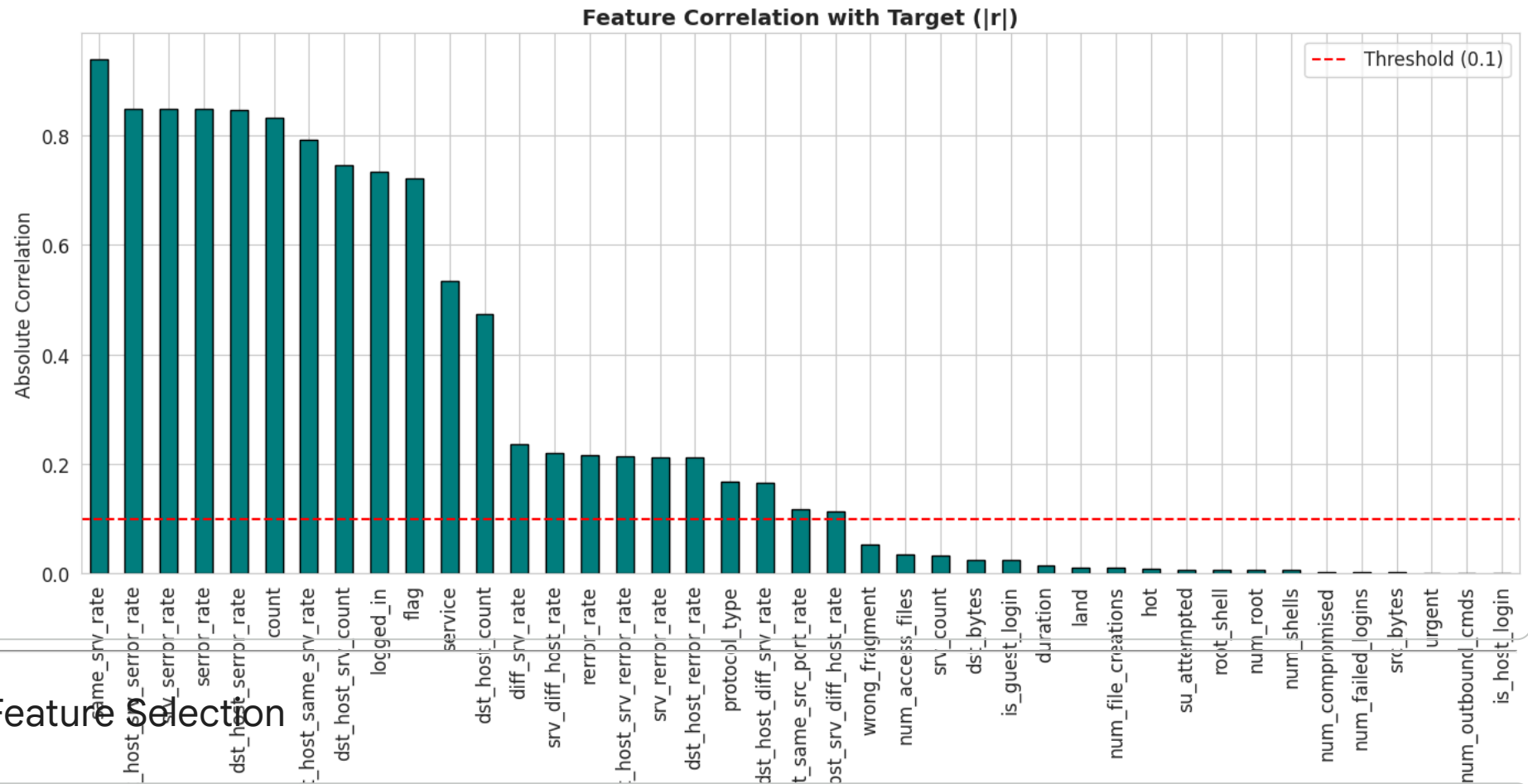
[Generate code with high_corr_df](#)

[New interactive sheet](#)

```
# Correlation of each feature with the target variable
target_corr = X.corrwith(y).abs().sort_values(ascending=False)

plt.figure(figsize=(14, 8))
target_corr.plot(kind='bar', color='teal', edgecolor='black')
plt.title('Feature Correlation with Target ( $|r|$ )', fontsize=14, fontweight='bold')
plt.ylabel('Absolute Correlation')
plt.axhline(y=0.1, color='red', linestyle='--', label='Threshold (0.1)')
plt.legend()
plt.tight_layout()
plt.show()

print("\nTop 15 features by correlation with target:")
print(target_corr.head(15))
```

< 5. Feature Selection

```
# Method 1: Remove highly correlated features (keep one from each correlated pair)
# Identify features to drop from highly correlated pairs
features_to_drop = set()
for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > 0.95:
            # Drop the feature with lower correlation to target
            feat_i = corr_matrix.columns[i]
            feat_j = corr_matrix.columns[j]
            corr_i = abs(X[feat_i].corr(y))
            corr_j = abs(X[feat_j].corr(y))
            if corr_i < corr_j:
                features_to_drop.add(feat_i)
            else:
                features_to_drop.add(feat_j)

print(f"Features to drop (corr > 0.95): {len(features_to_drop)}")
print(sorted(features_to_drop))
```

Features to drop (corr > 0.95): 7

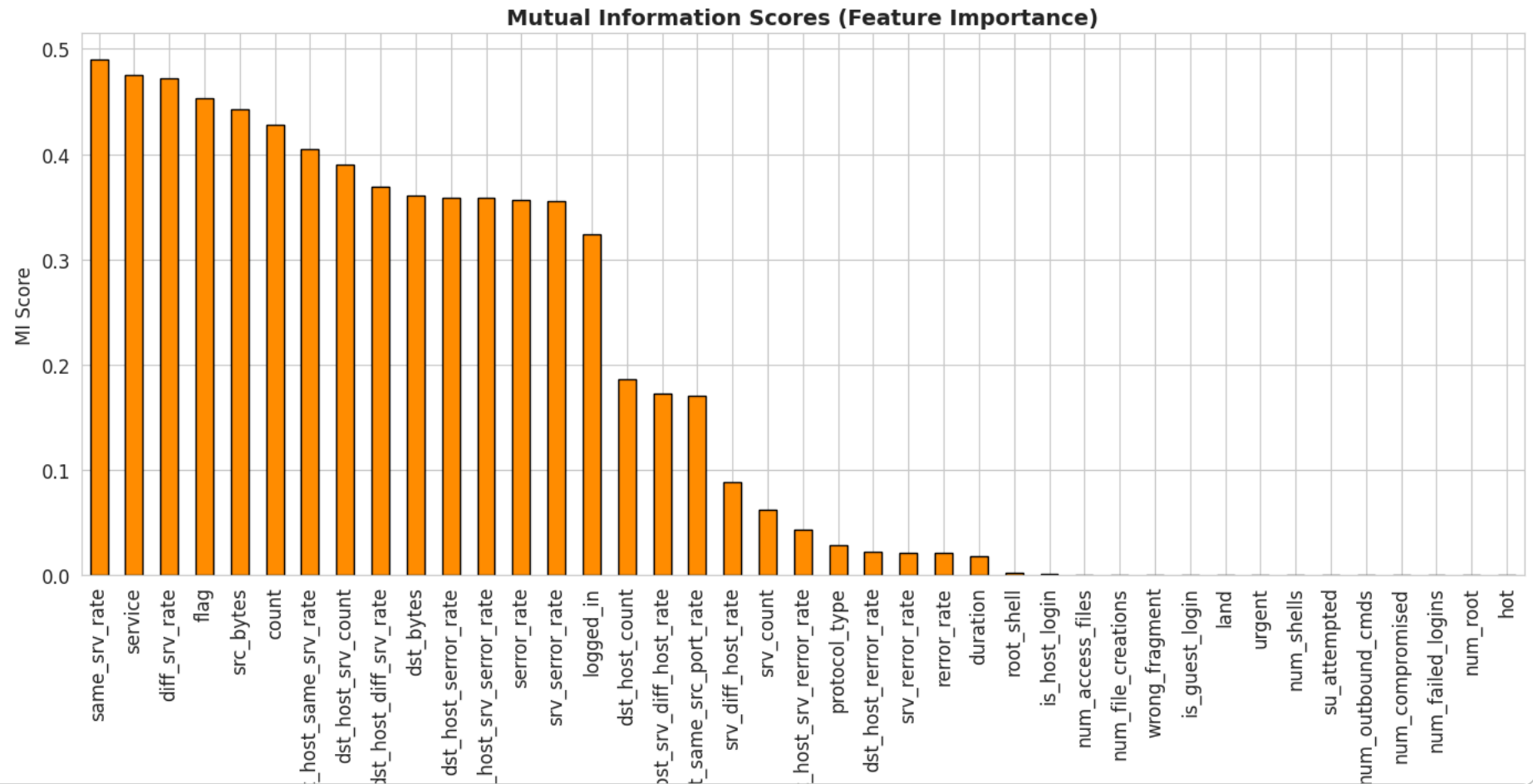
['dst_host_rerror_rate', 'dst_host_serror_rate', 'dst_host_srv_rerror_rate', 'num_compromised', 'serror_rate', 'srv_rerr

```
# Method 2: SelectKBest using mutual information
selector = SelectKBest(score_func=mutual_info_classif, k='all')
selector.fit(X, y)

mi_scores = pd.Series(selector.scores_, index=X.columns).sort_values(ascending=False)

plt.figure(figsize=(14, 8))
mi_scores.plot(kind='bar', color='darkorange', edgecolor='black')
plt.title('Mutual Information Scores (Feature Importance)', fontsize=14, fontweight='bold')
plt.ylabel('MI Score')
plt.tight_layout()
plt.show()

print("\nTop 20 features by Mutual Information:")
print(mi_scores.head(20))
```

```
# Select top K features based on mutual information
K = 25 # Number of features to select
selected_features = mi_scores.head(K).index.tolist()

# Also remove highly correlated features from the selected set
selected_features = [f for f in selected_features if f not in features_to_drop]

print(f"Final selected features ({len(selected_features)}):")
for i, f in enumerate(selected_features, 1):
    print(f" {i:2d}. {f} (MI: {mi_scores[f]:.4f})")

X_selected = X[selected_features].copy()
```

```
Final selected features (19):
1. same_srv_rate (MI: 0.4904)
2. service (MI: 0.4757)
3. diff_srv_rate (MI: 0.4716)
4. flag (MI: 0.4536)
5. src_bytes (MI: 0.4432)
6. count (MI: 0.4280)
7. dst_host_same_srv_rate (MI: 0.4053)
8. dst_host_srv_count (MI: 0.3904)
9. dst_host_diff_srv_rate (MI: 0.3694)
10. dst_bytes (MI: 0.3609)
11. dst_host_srv_serror_rate (MI: 0.3582)
12. logged_in (MI: 0.3236)
13. dst_host_count (MI: 0.1864)
14. dst_host_srv_diff_host_rate (MI: 0.1727)
15. dst_host_same_src_port_rate (MI: 0.1702)
16. srv_diff_host_rate (MI: 0.0882)
17. srv_count (MI: 0.0619)
18. protocol_type (MI: 0.0287)
19. rerror_rate (MI: 0.0208)
```

```
# Feature scaling
scaler = StandardScaler()
X_scaled = pd.DataFrame(
    scaler.fit_transform(X_selected),
    columns=X_selected.columns
)

print(f"Scaled feature matrix shape: {X_scaled.shape}")
print(f"\nScaled features summary:")
X_scaled.describe().loc[['mean', 'std', 'min', 'max']]
```

Scaled feature matrix shape: (100000, 19)

Scaled features summary:

	same_srv_rate	service	diff_srv_rate	flag	src_bytes	count	dst_host_same_srv_rate	dst_
mean	4.293099e-16	-1.076472e-16	1.630696e-17	-1.758593e-17	3.552714e-20	1.961098e-17	3.142375e-16	
std	1.000005e+00	1.000005e+00	1.000005e+00	1.000005e+00	1.000005e+00	1.000005e+00	1.000005e+00	
min	-1.989668e+00	-2.312916e+00	-2.708730e-01	-3.124823e+00	-1.872775e-02	-5.797896e-01	-1.572773e+00	
max	5.650531e-01	2.707794e+00	8.284003e+00	9.693752e-01	2.583545e+02	5.429998e+00	7.738460e-01	

```
# Train-Test Split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set: {X_train.shape[0]:,} samples")
print(f"Test set: {X_test.shape[0]:,} samples")
print(f"\nTraining target distribution:\n{y_train.value_counts().rename({0: 'Normal', 1: 'Attack'})}")
print(f"\nTest target distribution:\n{y_test.value_counts().rename({0: 'Normal', 1: 'Attack'})}")
```

```
Training set: 80,000 samples
Test set:    20,000 samples

Training target distribution:
binary_label
Normal      60490
Attack      19510
Name: count, dtype: int64

Test target distribution:
binary_label
Normal      15122
Attack       4878
Name: count, dtype: int64
```

✓ 6. Model Building & Training

We train six models as required:

1. **Gaussian Naive Bayes** — probabilistic classifier assuming feature independence
2. **Decision Tree** — rule-based splitting classifier
3. **Random Forest** — ensemble of decision trees with bagging
4. **Support Vector Machine (SVM)** — maximum-margin classifier
5. **Logistic Regression** — linear model for binary classification
6. **Gradient Boosting** — sequential ensemble that corrects errors iteratively

```
# Define all models
models = {
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier(
        max_depth=15, min_samples_split=10, random_state=42
    ),
    'Random Forest': RandomForestClassifier(
        n_estimators=100, max_depth=20, min_samples_split=5,
        n_jobs=-1, random_state=42
    ),
    'SVM': SVC(
        kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42
    ),
    'Logistic Regression': LogisticRegression(
        max_iter=1000, solver='lbfgs', random_state=42
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1, max_depth=5, random_state=42
    )
}

print(f"Models to train: {len(models)}")
for name in models:
    print(f"  - {name}")
```

```
Models to train: 6
- Naive Bayes
- Decision Tree
- Random Forest
- SVM
- Logistic Regression
- Gradient Boosting
```

```
# Train all models and collect metrics
results = {}

for name, model in models.items():
    print(f"\n{'='*60}")
    print(f"Training: {name}")
    print(f"{'='*60}")

    # Train
    start_time = time.time()
    model.fit(X_train, y_train)
    train_time = time.time() - start_time

    # Predict
    start_time = time.time()
    y_pred = model.predict(X_test)
    predict_time = time.time() - start_time

    # Probabilities for ROC-AUC
    if hasattr(model, 'predict_proba'):
        y_prob = model.predict_proba(X_test)[:, 1]
    else:
        y_prob = model.decision_function(X_test)

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    roc_auc = roc_auc_score(y_test, y_prob)

    results[name] = {
        'model': model,
        'y_pred': y_pred,
        'y_prob': y_prob,
        'accuracy': acc,
        'precision': prec,
```

```
        'recall': rec,
        'f1_score': f1,
        'roc_auc': roc_auc,
        'train_time': train_time,
        'predict_time': predict_time
    }

    print(f" Accuracy:      {acc:.4f}")
    print(f" Precision:    {prec:.4f}")
    print(f" Recall:         {rec:.4f}")
    print(f" F1-Score:       {f1:.4f}")
    print(f" ROC-AUC:        {roc_auc:.4f}")
    print(f" Train time:     {train_time:.2f}s")
    print(f" Predict time:   {predict_time:.4f}s")

    print(f"\n{'='*60}")
    print("All models trained successfully!")
    print(f"{'='*60}")
```

```
=====
Training: Decision Tree
=====
Accuracy:    0.9988
Precision:   0.9988
Recall:      0.9988
F1-Score:    0.9988
ROC-AUC:     0.9994
```



```
Predict time: 0.0999s

=====
Training: SVM
=====
Accuracy:    0.9967
Precision:   0.9967
Recall:      0.9967
F1-Score:    0.9967
ROC-AUC:     0.9996
Train time:  59.56s
Predict time: 1.0675s

=====
Training: Logistic Regression
=====
Accuracy:    0.9906
Precision:   0.9906
Recall:      0.9906
F1-Score:    0.9906
ROC-AUC:     0.9974
Train time:  0.60s
Predict time: 0.0043s

=====
Training: Gradient Boosting
=====
Accuracy:    0.9993
Precision:   0.9994
Recall:      0.9993
F1-Score:    0.9994
ROC-AUC:     0.9999
Train time:  22.60s
Predict time: 0.0360s

=====
All models trained successfully!
=====
```

7. Validation & Comparison

```
# Summary comparison table
comparison_data = []
for name, res in results.items():
    comparison_data.append({
        'Model': name,
        'Accuracy': res['accuracy'],
        'Precision': res['precision'],
        'Recall': res['recall'],
        'F1-Score': res['f1_score'],
        'ROC-AUC': res['roc_auc'],
        'Train Time (s)': res['train_time'],
        'Predict Time (s)': res['predict_time']
    })

comparison_df = pd.DataFrame(comparison_data).set_index('Model')
comparison_df = comparison_df.sort_values('F1-Score', ascending=False)

print("\n" + "=" * 80)
print("MODEL COMPARISON SUMMARY")
print("=" * 80)
comparison_df.style.format({
    'Accuracy': '{:.4f}', 'Precision': '{:.4f}', 'Recall': '{:.4f}',
    'F1-Score': '{:.4f}', 'ROC-AUC': '{:.4f}',
    'Train Time (s)': '{:.2f}', 'Predict Time (s)': '{:.4f}'
})
```

```
=====
MODEL COMPARISON SUMMARY
=====
```

	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Train Time (s)	Predict Time (s)
Model							
Gradient Boosting	0.9993	0.9994	0.9993	0.9994	0.9999	22.60	0.0360
Random Forest	0.9993	0.9993	0.9993	0.9993	1.0000	4.99	0.0999
Decision Tree	0.9988	0.9988	0.9988	0.9988	0.9994	0.58	0.0043
SVM	0.9967	0.9967	0.9967	0.9967	0.9996	59.56	1.0675
Logistic Regression	0.9906	0.9906	0.9906	0.9906	0.9974	0.60	0.0043
Naive Bayes	0.9786	0.9788	0.9786	0.9787	0.9916	0.04	0.0063

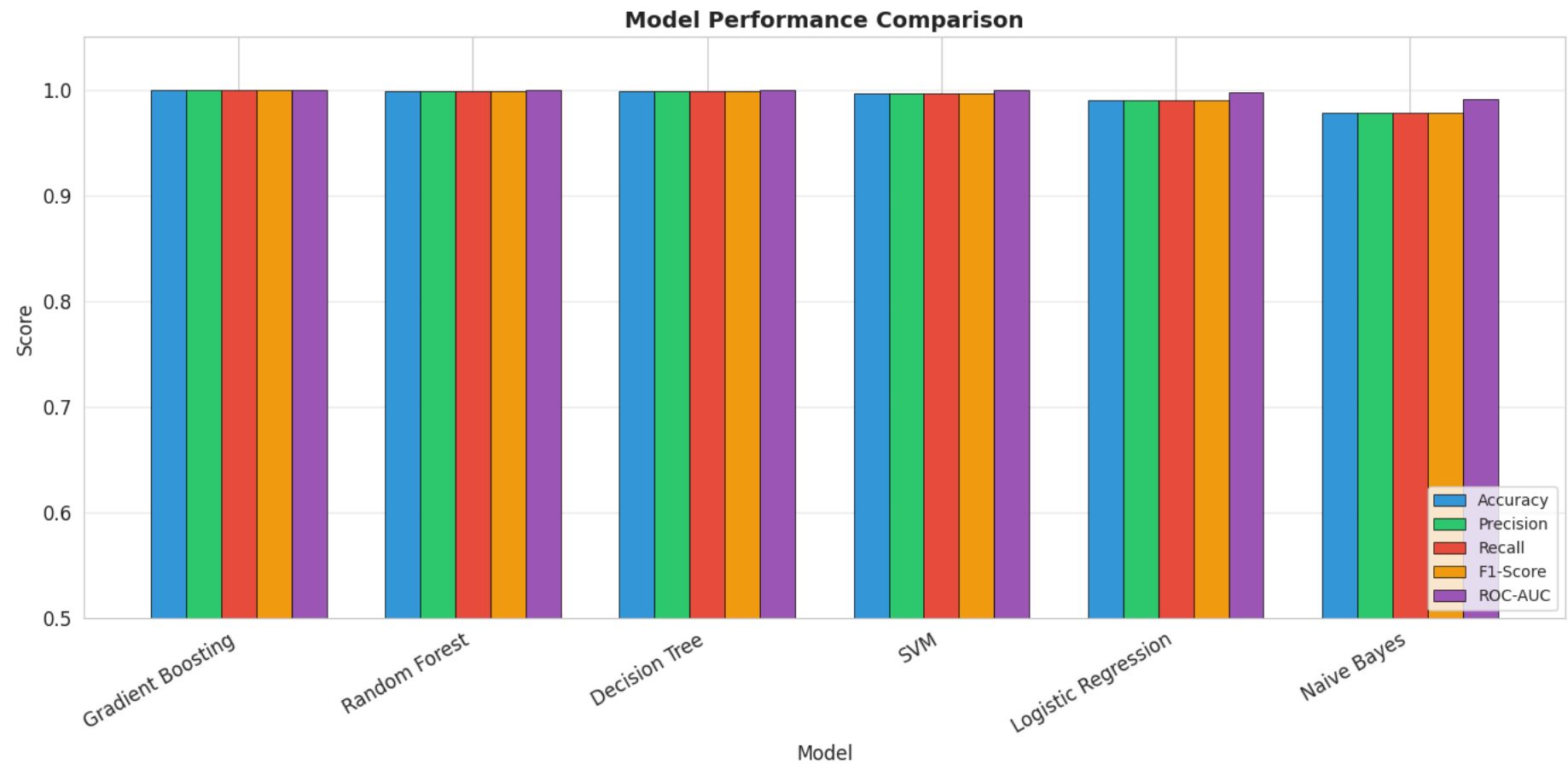
```
# Visual comparison of metrics
metrics_to_plot = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']
plot_df = comparison_df[metrics_to_plot]

fig, ax = plt.subplots(figsize=(14, 7))
x = np.arange(len(plot_df.index))
width = 0.15
colors = ['#3498db', '#2ecc71', '#e74c3c', '#f39c12', '#9b59b6']

for i, metric in enumerate(metrics_to_plot):
    bars = ax.bar(x + i * width, plot_df[metric], width,
                  label=metric, color=colors[i], edgecolor='black', linewidth=0.5)

ax.set_xlabel('Model', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Model Performance Comparison', fontsize=14, fontweight='bold')
ax.set_xticks(x + width * 2)
ax.set_xticklabels(plot_df.index, rotation=30, ha='right')
ax.legend(loc='lower right', fontsize=10)
ax.set_ylim(0.5, 1.05)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# Confusion matrices for all models
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

for idx, (name, res) in enumerate(results.items()):
    cm = confusion_matrix(y_test, res['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
                xticklabels=['Normal', 'Attack'], yticklabels=['Normal', 'Attack'])
    axes[idx].set_title(f'{name}\n(Acc: {res["accuracy"]:.4f})', fontweight='bold')
    axes[idx].set_ylabel('Actual')
    axes[idx].set_xlabel('Predicted')

plt.suptitle('Confusion Matrices – All Models', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

